

Software Design

Chapter 5 — Source-Grounded Study Notes

Course: Advanced Software Engineering (CS603), 3 Credit Hours

Instructor: Asst. Prof. Dr. Ali Fahim Ni'ma

Institution: College of Computer Science & Information Technology, University of Wasit

Source: Aggarwal & Singh, Software Engineering, 3rd ed., Ch.5 companion slides (99 slides)

Method: Every paragraph anchored to its source slide as [S:n]

Status: Draft for review — verify anchors before final use

Table of Contents

Update field (right-click > Update Field / F9)

(In Word: select all and press F9, or right-click > Update Field.)

List of Figures

Figure 1. Figure 1. The design framework of the source [S:3, Fig.1]: requirements are gathered and analysed, a high-level design is conceived, refined and documented, then validated against the requirements.

Figure 2. Figure 2. The two-part design process [S:5, Fig.2]: conceptual design answers the customer's "what"; technical design answers the builders' "how".

Figure 3. Figure 3. Modularity and software cost [S:14, Fig.4]: the source's curve of cost of effort against the number of modules, with the marked region of minimum cost.

Figure 4. Figure 4. High versus low coupling [S:15–16, Fig.5]: more dependency links between modules mean tighter coupling.

Figure 5. Figure 5. The types of module coupling, from worst (content) to best (data) [S:19, Fig.7].

Figure 6. Figure 6. Example of common coupling [S:22, Fig.8]: modules share a global data area, so a change to the common data affects every module.

Figure 7. Figure 7. Types of module cohesion, weakest to strongest [S:27, Fig.11].

Figure 8. Figure 8. Structure chart for "update file" [S:43, Fig.18]: modules in a hierarchy with labelled data couples (source notation in Fig.17 [S:42]).

Figure 9. Figure 9. Inheritance [S:68, Fig.23]: subclasses Square and Triangle inherit state and behaviour from the base class Shape.

Figure 10. Figure 10. Steps for analysis and design of an object-oriented system [S:71–72, Fig.25].

Figure 11. Figure 11. Sequence diagram for the "issue book" scenario of the library case [S:86, modelled from S:81–83].

Software Design — Chapter 5

Grounded study document reconstructed from the source slide deck

(K. K. Aggarwal & Y. Singh, Software Engineering, 3rd ed., Chapter 5 — companion slides, 99 slides).

Every paragraph carries a source anchor of the form [S:n], where n is the slide number in the source deck.

No statement appears here that is not readable in the source; where the source is a figure, the figure is cited as [S:n, Fig.x].

1. What Is Design? [S:2]

Design is presented in the source as an activity that is **more creative than analysis** and that is fundamentally a **problem-solving activity** [S:2]. The distinction the source draws is one of question: analysis establishes *what* is required, whereas design answers *how* the requirement will be met — the source labels design directly as the **How** [S:2].

The concrete output of the design activity is the **Software Design Document (SDD)** [S:2]. The source also states the audience the design must serve: a design is produced to satisfy both the **customer** and the **developers (implementers)** [S:4]. This two-audience requirement explains why design is later split into a conceptual part aimed at the customer and a technical part aimed at the builders [S:5].

2. The Design Framework [S:3, Fig.1]

The source expresses design as a framework of ordered activities rather than a single step [S:3, Fig.1]. It begins from the **initial requirements**, then: *gather data on user requirements; analyse requirements data; conceive of a high-level design; refine and document the design; obtain answers to requirement questions; and validate the design against the requirements*, ending in the **completed design** [S:3, Fig.1].

Two features of the framework are worth noting because they recur later in the chapter. First, the path from requirements to a completed design passes through a **high-level design** before any refinement [S:3, Fig.1] — this is the seed of the top-down strategy discussed in section 9 [S:34]. Second, the framework contains a feedback step: requirement questions are answered and the design is validated against the requirements, so the process can loop rather than terminate after one pass [S:3, Fig.1].

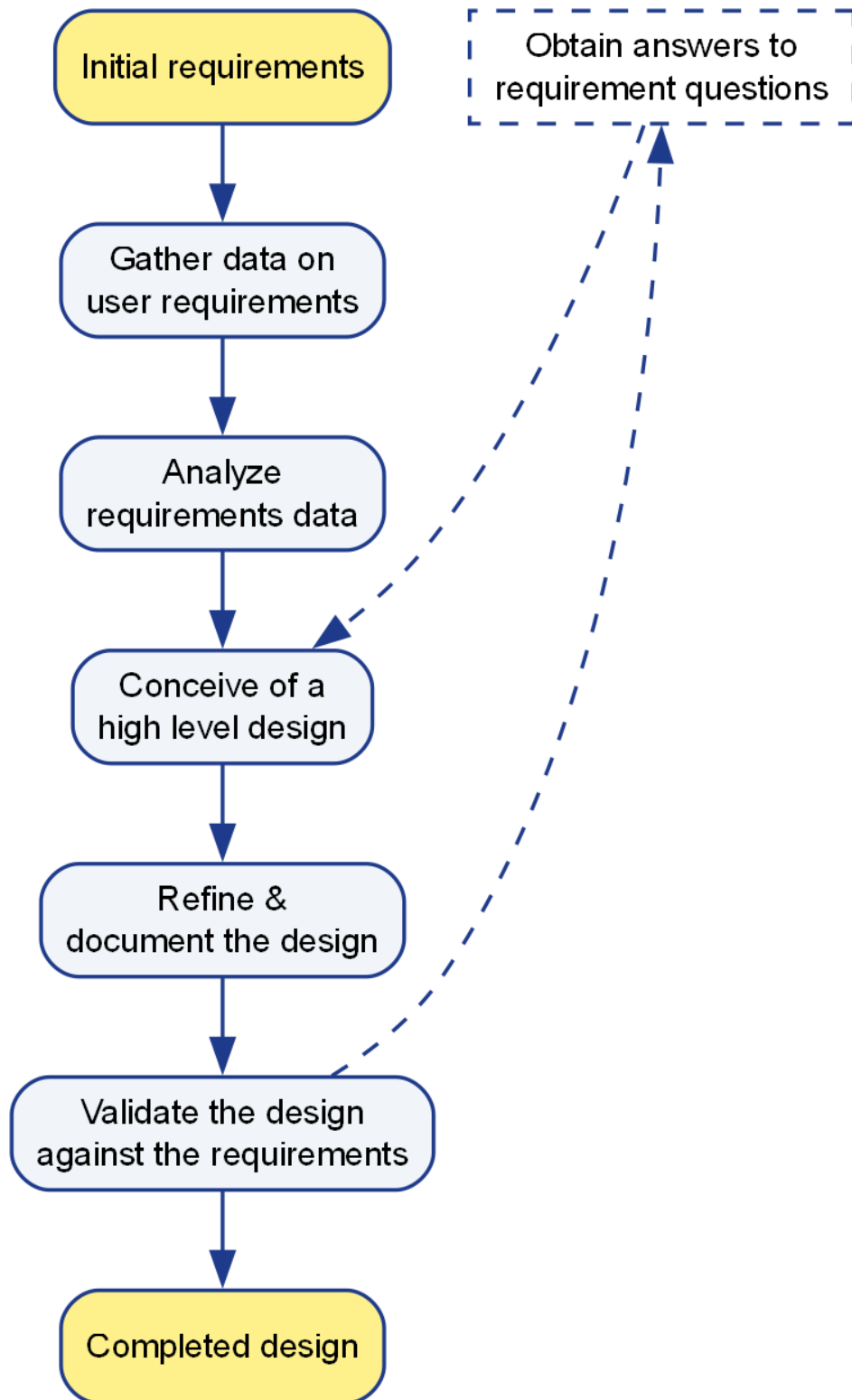


Figure 1. The design framework of the source [S:3, Fig.1]: requirements are gathered and analysed, a high-level design is conceived, refined and documented, then validated against the requirements.

3. Conceptual Design and Technical Design [S:5–7]

The source describes design as a **two-part process** [S:5, Fig.2]. The two parts sit between the **customer** and the **system builders**, with the designers in the middle: the customer's view is the *what* and becomes **conceptual design**; the builders' view is the *how* and becomes **technical design** [S:5, Fig.2].

3.1 What conceptual design answers [S:6]

Conceptual design is defined by the set of questions it answers, quoted from the source [S:6]:

- Where will the data come from?
- What will happen to data in the system?
- How will the system look to users?
- What choices will be offered to users?
- What is the timings of events?
- How will the reports and screens look like?

3.2 What technical design describes [S:7]

Technical design describes the following, again quoted from the source [S:7]: **hardware configuration; software needs; communication interfaces; I/O of the system; software architecture; network architecture**; and, in the source's own summary, *any other thing that translates the requirements into a solution to the customer's problem* [S:7].

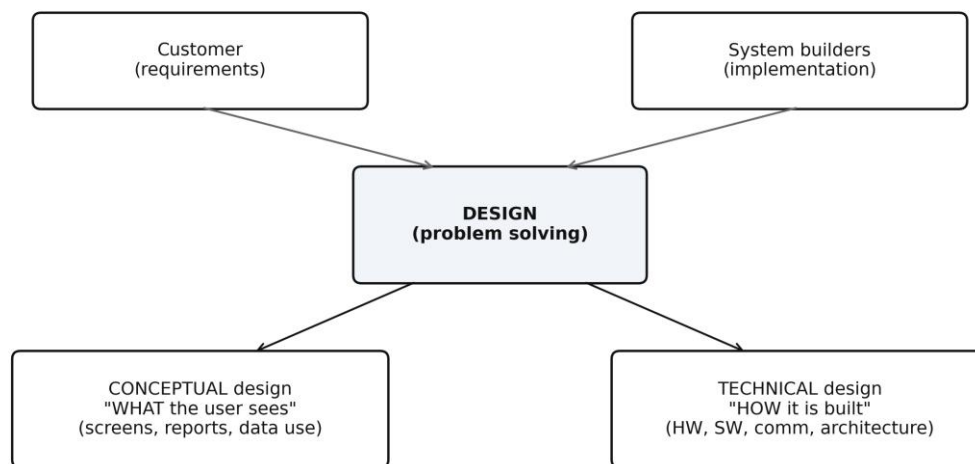


Figure 2. The two-part design process [S:5, Fig.2]: conceptual design answers the customer's "what"; technical design answers the builders' "how".

4. Qualities of a Design and Levels of Formality [S:8–9]

4.1 Required qualities [S:8]

The source states that a design needs to be **correct and complete, understandable, at the right level, and maintainable** [S:8]. These four qualities are the chapter's stated acceptance criteria for a design.

4.2 From informal outline to finished design [S:9, Fig.3]

Design is not produced in its final form in one step. The source shows a transformation of increasing formality [S:9, Fig.3]: an **informal design outline**, then an **informal design**, then a **more formal design**, and finally the **finished design** [S:9, Fig.3]. This progression is answered directly by exercise 5.2, which asks how an informal design is transformed into a detailed design [S:97].

5. Modularity [S:10–14]

5.1 What a module is [S:10–11]

The source begins by refusing a single definition: the term *module* ranges across the following, and **all of these definitions are correct** [S:10–11]:

- Fortran subroutine
- Ada package
- Procedures and functions of Pascal and C
- C++ / Java classes
- Java packages
- Work assignment for an individual programmer

The unifying statement the source then gives is that **a modular system consists of well-defined, manageable units with well-defined interfaces among the units** [S:11].

5.2 Properties of a modular system [S:12]

The source lists six properties [S:12]: a **well-defined subsystem**; a **well-defined purpose**; the module **can be separately compiled and stored in a library**; a module **can use other modules**; a module **should be easier to use than to build**; and a module is **simpler from the outside than from the inside** [S:12]. Exercise 5.4 asks precisely for this list [S:97].

5.3 Why modularity matters [S:13]

The source states that modularity **is the single attribute of software that allows a program to be intellectually manageable**, and that it **enhances design clarity**, which in turn **eases implementation, debugging, testing, documenting, and maintenance** of the software product [S:13].

5.4 Modularity and cost [S:14, Fig.4]

Figure 4 of the source is titled "**Modularity and software cost**" [S:14, Fig.4]. The figure plots **cost of effort** against the **number of modules** and distinguishes the **cost to integrate** from the **total software cost**, marking a **region of minimum cost** over the number of modules M [S:14, Fig.4]. The figure therefore establishes that modularity has an optimum rather than being "more is always better".

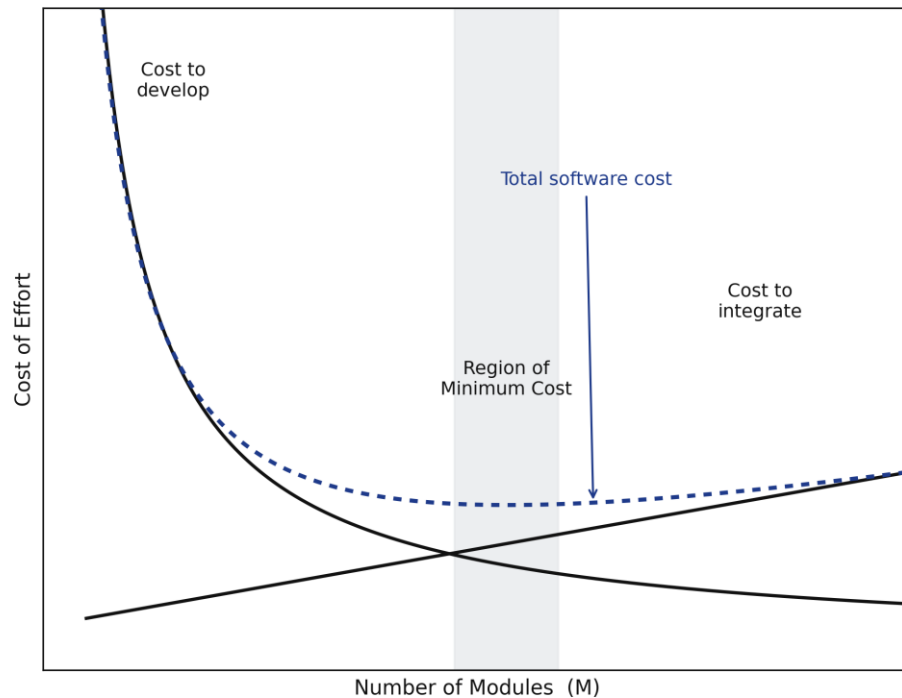


Figure 3. Modularity and software cost [S:14, Fig.4]: the source's curve of cost of effort against the number of modules, with the marked region of minimum cost.

6. Module Coupling [S:15–24]

6.1 Definition [S:15]

Coupling is the measure of the degree of interdependence between modules [S:15]. The source illustrates three degrees with Figure 5 [S:15–16, Fig.5]: an **uncoupled** pair with no dependencies (a); a **loosely coupled** pair with some dependencies (b); and a **highly coupled** pair with many dependencies (c) [S:15–16, Fig.5].

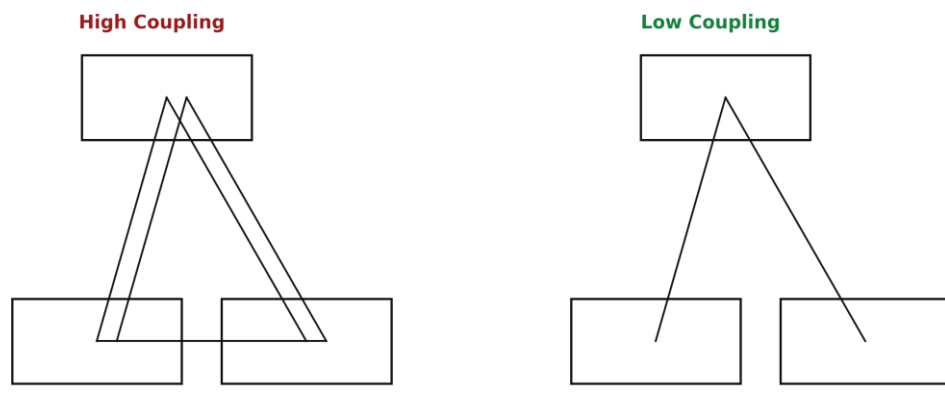


Figure 4. High versus low coupling [S:15–16, Fig.5]: more dependency links between modules mean tighter coupling.

6.2 How low coupling is achieved [S:17]

The source gives four concrete means of keeping coupling low [S:17]: **controlling the number of parameters passed amongst modules**; **avoiding passing undesired data to the calling module**; **maintaining a parent/child relationship between calling and called modules**; and **passing data, not control information** [S:17].

6.3 Worked example: editing a student record [S:18, Fig.6]

The source illustrates coupling with an "Edit student record" / "Retrieve student record" pair [S:18, Fig.6]. In the **poor design (tight coupling)**, the student name, student ID, address and course are passed between the modules together with the student record and an EOF flag. In the **good design (loose coupling)**, only the student ID is passed and the record is returned [S:18, Fig.6]. The example is the source's own demonstration that passing the minimum data reduces coupling.

6.4 The types of coupling, worst to best [S:19, Fig.7]

The source orders the ways two procedures A and B may be coupled from worst to best [S:19, Fig.7]: **Content coupling**, **Common coupling**, **External coupling**, **Control coupling**, **Stamp coupling**, and **Data coupling** [S:19, Fig.7]. This single ordering is the chapter's most examinable list.

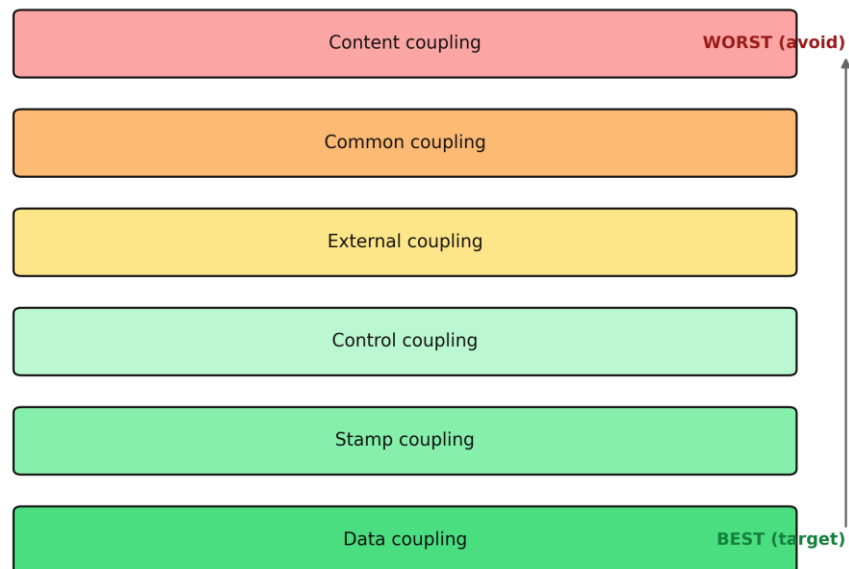


Figure 5. The types of module coupling, from worst (content) to best (data) [S:19, Fig.7].

6.5 Definitions of each coupling type [S:20–24]

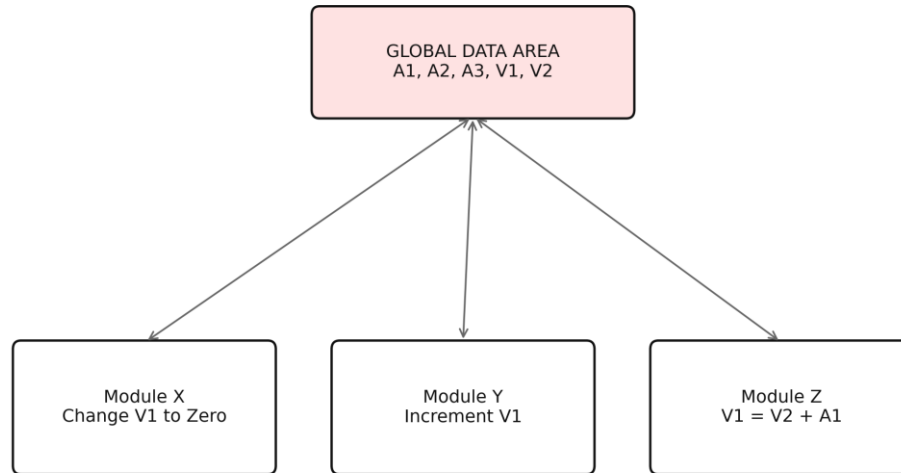
The source defines the types as follows.

Data coupling. Modules A and B are data coupled if **their dependency is based on the fact that they communicate by only passing of data**; other than communicating through data, the two modules are independent [S:20].

Stamp coupling. Stamp coupling occurs between A and B **when a complete data structure is passed from one module to another** [S:20].

Control coupling. A and B are control coupled if they communicate by passing of control information, usually accomplished by means of flags that are set by one module and reacted upon by the dependent module [S:21].

Common coupling. With common coupling, A and B have shared data; global data areas are commonly found in programming languages, and making a change to the common data means tracing back to all the modules which access that data to evaluate the effect of changes [S:21]. Figure 8 of the source illustrates this with a shared global data area [S:22, Fig.8].



All modules read/write the same global variables -> changes ripple everywhere

Figure 6. Example of common coupling [S:22, Fig.8]: modules share a global data area, so a change to the common data affects every module.

Content coupling. Content coupling occurs when module A changes the data of module B, or when control is passed from one module to the middle of another; in the source's Figure 9, module B branches into D even though D is supposed to be under the control of C [S:23–24, Fig.9].

6.6 The objective [S:31]

The combined goal stated by the source is that a software engineer must design the modules with the goal of high cohesion and low coupling [S:31]. That objective is treated in section 8 below together with cohesion.

7. Module Cohesion [S:25–31]

7.1 Definition [S:25]

Cohesion is a measure of the degree to which the elements of a module are functionally related [S:25]. Figure 10 of the source depicts cohesion as the "strength of relations within modules" [S:25, Fig.10] — that is, cohesion faces inward while coupling faces outward between modules.

7.2 The types of cohesion [S:26–27]

The source first lists the types as: **functional**, **sequential**, **procedural**, **temporal**, **logical**, and **coincident** cohesion [S:26]. Figure 11 orders them as a ladder from **worst (low)** to **best (high)** [S:27, Fig.11]:

Coincidental → Logical → Temporal → Procedural → Communicational → Sequential → Functional
[S:27, Fig.11].

Note that the ordered ladder in Figure 11 contains **seven** levels (it includes *communicational*, which the plain list on slide 26 omits); this discrepancy is in the source and is worth stating when revising [S:26–27].

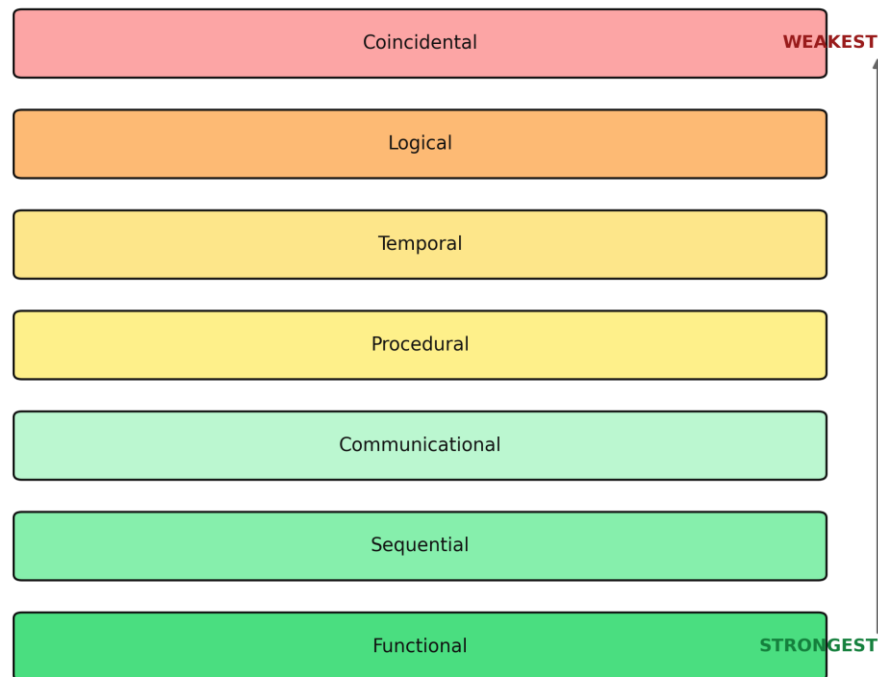


Figure 7. Types of module cohesion, weakest to strongest [S:27, Fig.11].

7.3 Definitions of each cohesion type [S:28–30]

Functional cohesion. A and B are **part of a single functional task**; this is, in the source's words, a very good reason for them to be contained in the same procedure [S:28]. Exercise 5.7 states the same definition — operations that are part of a single functional task and placed in the same procedures [S:95].

Sequential cohesion. Module A **outputs some data which forms the input to B**; this is the reason for them to be contained in the same procedure [S:28].

Procedural cohesion. Occurs in modules whose **instructions accomplish different tasks yet have been combined because there is a specific order in which the tasks are to be completed** [S:29]. Exercise 5.9 identifies this as the module whose instructions are related through flow of control [S:95].

Temporal cohesion. A module exhibits temporal cohesion when it **contains tasks that are related by the fact that all tasks must be executed in the same time-span** [S:29]. This is the source's answer to MCQ 5.6, "cohesion with respect to time" [S:95].

Logical cohesion. Occurs in modules that **contain instructions that appear to be related because they fall into the same logical class of functions** [S:30].

Coincidental cohesion. Exists in modules that **contain instructions that have little or no relationship to one another** [S:30]. It is the worst level of Figure 11 [S:27].

8. Relationship between Cohesion and Coupling [S:31, Fig.12]

The source closes the modularity discussion with a warning and a rule [S:31, Fig.12]. The warning: **if the software is not properly modularised, a host of seemingly trivial enhancements or changes will result in the death of the project** [S:31]. The rule, already quoted, is to design modules with the **goal of high cohesion and low coupling** [S:31].

9. Strategy of Design [S:32–35]

9.1 Why a strategy is needed [S:32]

The source states that a **good system design strategy is to organise the program modules in such a way that they are easy to develop and, later, to change** [S:32]. **Structured design techniques help developers deal with the size and complexity of programs**; analysts create instructions for developers about how code should be written and how pieces of code should fit together to form a program [S:32]. The source gives two reasons this is important [S:32]: first, **even pre-existing code needs to be understood, organised and pieced together**; second, **it is still common for the project team to have to write some code and produce original programs that support the application logic of the system** [S:32].

9.2 Bottom-up design [S:33, Fig.13]

In bottom-up design, the low-level modules **are collected together in the form of a "library"** [S:33, Fig.13].

9.3 Top-down design [S:34]

A top-down design **starts by identifying the major modules of the system, decomposing them into their lower-level modules, and iterating until the desired level of detail is achieved** [S:34]. The source names this **stepwise refinement**: starting from an abstract design, each step refines the design to a more concrete level, until a level is reached where **no more refinement is needed and the design can be implemented directly** [S:34].

9.4 Hybrid design [S:35]

The source argues that **for the top-down approach to be effective, some bottom-up approach is essential**, for three reasons [S:35]:

- to **permit common sub-modules**;
- near the **bottom of the hierarchy**, where intuition is simpler and the need for **bottom-up testing is greater**, because there are more modules at low levels than at high levels;
- in the **use of pre-written library modules**, in particular the **reuse of modules** [S:35].

Exercise 5.11 asks which strategy is most popular and practical; the source's own three-reason justification for combining the two directions is the material to answer it [S:35, S:98].

10. Function-Oriented Design [S:36–49]

10.1 Definition [S:36]

Function-oriented design is an approach to software design where the design is decomposed into a set of interacting units where each unit has a clearly defined function; the system is therefore designed from a functional viewpoint [S:36].

10.2 Top-down and reusable structures [S:38–39, Fig.14–15]

Refinement continues until we reach the statement level of the programming language, at which point the program can be described as a **tree of refinement** [S:38, Fig.14]. The source notes a trade-off: if a program is created top-down, the modules become very specialised — each module is used by at most one other module, its parent [S:39]. But a module may be required by several other modules, which gives the reusable structure of Figure 15 [S:39, Fig.15]. This is the source's own statement of the specialisation-versus-reuse trade-off.

10.3 Design notations [S:40]

For function-oriented design, the source says the design can be represented graphically or mathematically by the following notations [S:40]: **Data flow diagrams**; **Data dictionaries**; **Structure charts**; **Pseudocode** [S:40]. Exercise 5.15 asks for well-established function-oriented techniques, and this is the list [S:98].

10.4 Structure charts [S:41–43, Fig.16–18]

A structure chart partitions a system into black boxes, where a black box means that functionality is known to the user without knowledge of the internal design [S:41, Fig.16]. The source provides the chart notations in Figure 17 [S:42, Fig.17] and a worked structure chart for an "update file" task in Figure 18 [S:43, Fig.18].

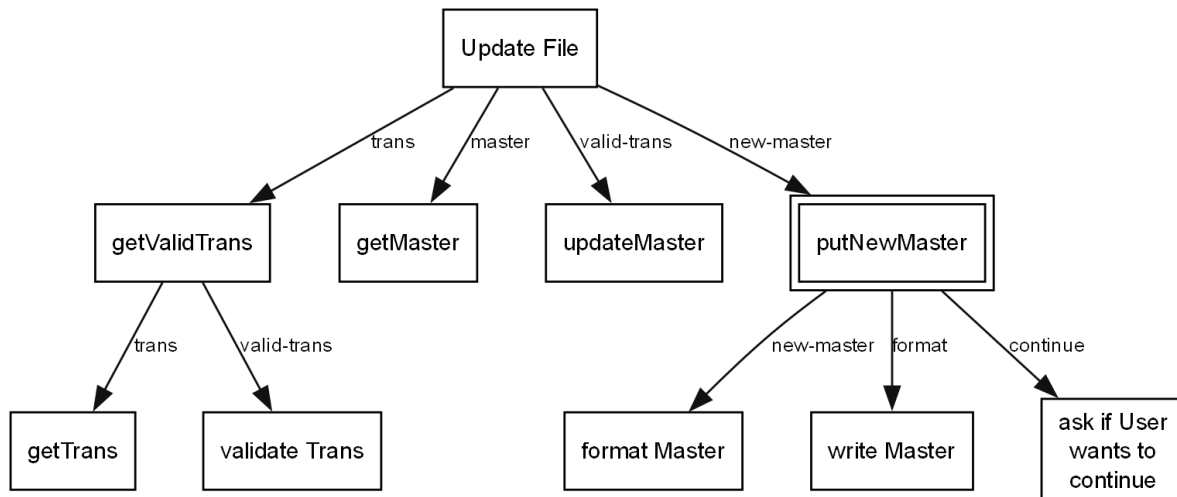


Figure 8. Structure chart for "update file" [S:43, Fig.18]: modules in a hierarchy with labelled data couples (source notation in Fig.17 [S:42]).

10.5 Transaction-centred structure [S:44–45, Fig.19]

A **transaction-centred structure** describes a system that processes a number of different types of transactions [S:44, Fig.19]. In the source's figure the **MAIN** module controls the system operation, and its

functions are [S:45]: **invoke the INPUT module to read a transaction; determine the kind of transaction and select one of a number of transaction modules to process that transaction; and output the results of the processing by calling the OUTPUT module** [S:45].

10.6 Pseudocode [S:46]

Pseudocode can be used **in both the preliminary and detailed design phases** [S:46]. Using pseudocode, **the designer describes system characteristics using short, concise, English-language phrases that are structured by key words such as If-Then-Else, While-Do, and End** [S:46].

10.7 Functional procedure layers [S:47–49]

The source states that **functions are built in layers**, with additional notation used to specify details [S:47]. The layers are:

- **Level 0** — function or procedure name; relationship to other system components (e.g. part of which system, called by which routines); a brief description of the function purpose; author and date [S:47].
- **Level 1** — function parameters (problem variables, types, purpose); global variables (problem variable, type, purpose, sharing information); routines called by the function; side effects; input/output assertions [S:48].
- **Level 2** — local data structures; timing constraints; exception handling (conditions, responses, events); any other limitations [S:49].
- **Level 3** — the **body** (structured chart, English pseudocode, decision tables, flow charts, etc.) [S:49].

11. Documenting the Design: IEEE Std 1016-1998 [S:50–57]

11.1 What an SDD is [S:50–52]

The source introduces the **IEEE recommended practice for software design descriptions (IEEE Std 1016-1998)** and defines an **SDD as a representation of a software system that is used as a medium for communicating software design information** [S:50]. The SDD's scope and references are given; its references are IEEE Std 830-1998 (software requirements specifications) and IEEE Std 610.12-1990 (glossary of software engineering terminology) [S:50].

The source then gives four definitions [S:51]: a **design entity** is *an element (component) of a design that is structurally and functionally distinct from other elements and that is separately named and referenced*; a **design view** is *a subset of design entity attribute information that is specifically suited to the needs of a software project activity*; **entity attributes** are *a named property or characteristic of a design entity*; and the **SDD** is *a representation of a software system created to facilitate analysis, planning, implementation and decision making* [S:51].

The **purpose** of the SDD is stated as follows: **the SDD shows how the software system will be structured to satisfy the requirements identified in the SRS; it is basically the translation of requirements into a description of the software structure, software components, interfaces, and data necessary for the implementation phase; hence the SDD becomes the blueprint for the implementation activity** [S:52]. Its design-description information content is **introduction, design entities, and design entity attributes** [S:52].

11.2 The entity attributes [S:53]

The source lists ten attributes and associated information items [S:53]: (a) **Identification**, (b) **Type**, (c) **Purpose**, (d) **Function**, (e) **Subordinates**, (f) **Dependencies**, (g) **Interface**, (h) **Resources**, (i) **Processing**, (j) **Data** [S:53].

11.3 Organisation and design views [S:54–57]

The source notes that each writer may have a different view of what are essential aspects of a design; the **organisation of the SDD is given in Table 1**, described as *one of the possible ways* to organise and format the SDD [S:54]. A recommended organisation of the SDD into separate **design views** to facilitate information access and assimilation is given in **Table 2** [S:54, S:57]. The four views and their attributes, quoted from Table 2, are [S:57]:

- **Decomposition description** — partition of the system into design entities; entity attributes: identification, type, purpose, function, subordinate; example representation: hierarchical decomposition diagram, natural language [S:57].
- **Dependency description** — description of relationships among entities and system resources; attributes: identification, type, purpose, dependencies, resources; example representation: structure chart, data flow diagrams, transaction diagrams [S:57].
- **Interface description** — a list of everything a designer, developer or tester needs to know to use the design entities that make up the system; attributes: identification, function, interfaces; example representation: interface files, parameter tables [S:57].
- **Detail description** — description of the internal design details of an entity; attributes: identification, processing, data; example representation: flow charts, PDL [S:57].

12. Object-Oriented Design [S:58–80]

12.1 Definition and relationship to function-oriented design [S:58–59]

Object-oriented design is the result of focusing attention not on the function performed by the program, but instead on the data that are to be manipulated by the program; the source states that it is therefore **orthogonal to function-oriented design** [S:58]. It begins with an examination of the real-world "things" that are part of the problem to be solved; these things (called **objects**) are characterised individually in terms of their **attributes** and **behaviour** [S:58].

The source adds that object-oriented design **is not dependent on any specific implementation language**, and that objects have two basic concepts [S:59]: **behaviour** (they do things) and **state** (which changes when they do things) [S:59].

12.2 The ten terms of object design [S:60–70]

The source enumerates ten related terms [S:60–70].

- Objects.** An object is *an entity able to save a state (information) and which offers a number of operations (behaviour) to either examine or affect this state*; an object is characterised by a number of operations and a state which remembers the effect of these operations [S:60].
- Messages.** Objects **communicate by message passing**; a message consists of the **identity of the target object**, the **name of the requested operation**, and any other information needed; messages are **often implemented as procedure or function calls** [S:61].

iii. **Abstraction.** In object-oriented design, **complexity is managed using abstraction**; abstraction is **the elimination of the irrelevant and the amplification of the essentials** [S:61].

iv. **Class.** Some objects share common characteristics and can be grouped; this grouping is a **class**, defined as **a set of objects that share a common structure and a common behaviour** [S:62]. The source's example is a class **car** whose instances are **Indica, Santro, Maruti, Indigo** [S:62, Fig.20]; classes are useful because they **act as a blueprint for objects**, and the source uses the **square class** to show the representation [S:62, Fig.21].

v. **Attributes.** An **attribute is a data value held by the objects in a class**; the square class has two attributes — a **colour** and an **array of points** — and each attribute has a value for each object instance; attributes are shown as the **second part of the class** [S:65, Fig.21].

vi. **Operations.** An **operation is a function or transformation that may be applied to or by objects in a class**; the square class has two operations, **set colour()** and **draw()**; all objects in a class share the same operations, and operations are shown in the **third part of the class** [S:65, Fig.21].

vii. **Inheritance.** Introducing a **triangle** class alongside the square class [S:66, Fig.22], the source observes that both can be abstracted at a high level as **shapes** that "draw themselves"; the important parts are brought into a new class called **Shape** [S:67, Fig.23]. This sort of abstraction is **called inheritance**: the low-level classes (subclasses or **derived classes**) **inherit state and behaviour from the high-level class** (superclass or **base class**) [S:68].

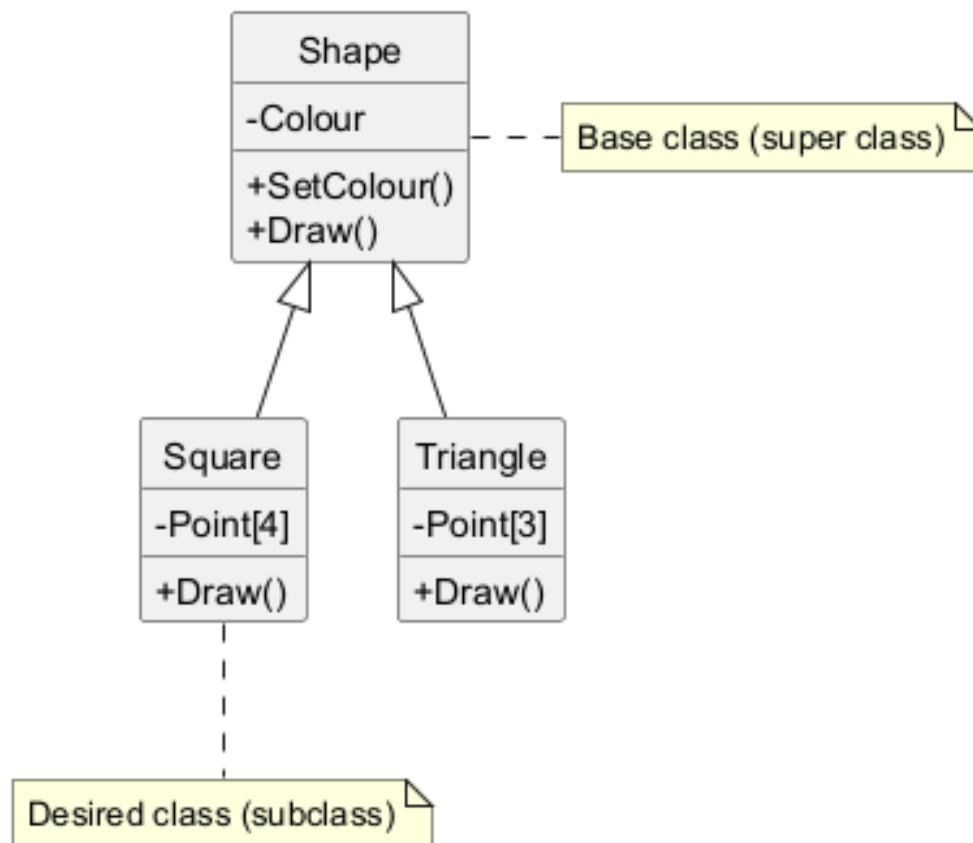


Figure 9. Inheritance [S:68, Fig.23]: subclasses **Square** and **Triangle** inherit state and behaviour from the base class **Shape**.

viii. **Polymorphism.** When we **abstract just the interface of an operation and leave the implementation to subclasses**, it is called a **polymorphic operation** and the process is called **polymorphism** [S:69].

ix. **Encapsulation (information hiding).** Encapsulation consists of the separation of the external aspects of an object from the internal implementation details of the object [S:69].

x. **Hierarchy.** Hierarchy involves organising something according to some particular order or rank; it is another mechanism for reducing the complexity of software by being able to treat and express sub-types in a generic way [S:69, Fig.24].

12.3 Steps to analyse and design an object-oriented system [S:71–80, Fig.25]

The source lays out a sequence of steps (Figure 25) [S:71–72]:

i. **Create use-case model.** The first step is to identify the actors interacting with the system, then write the use case and draw the use-case diagram [S:73].

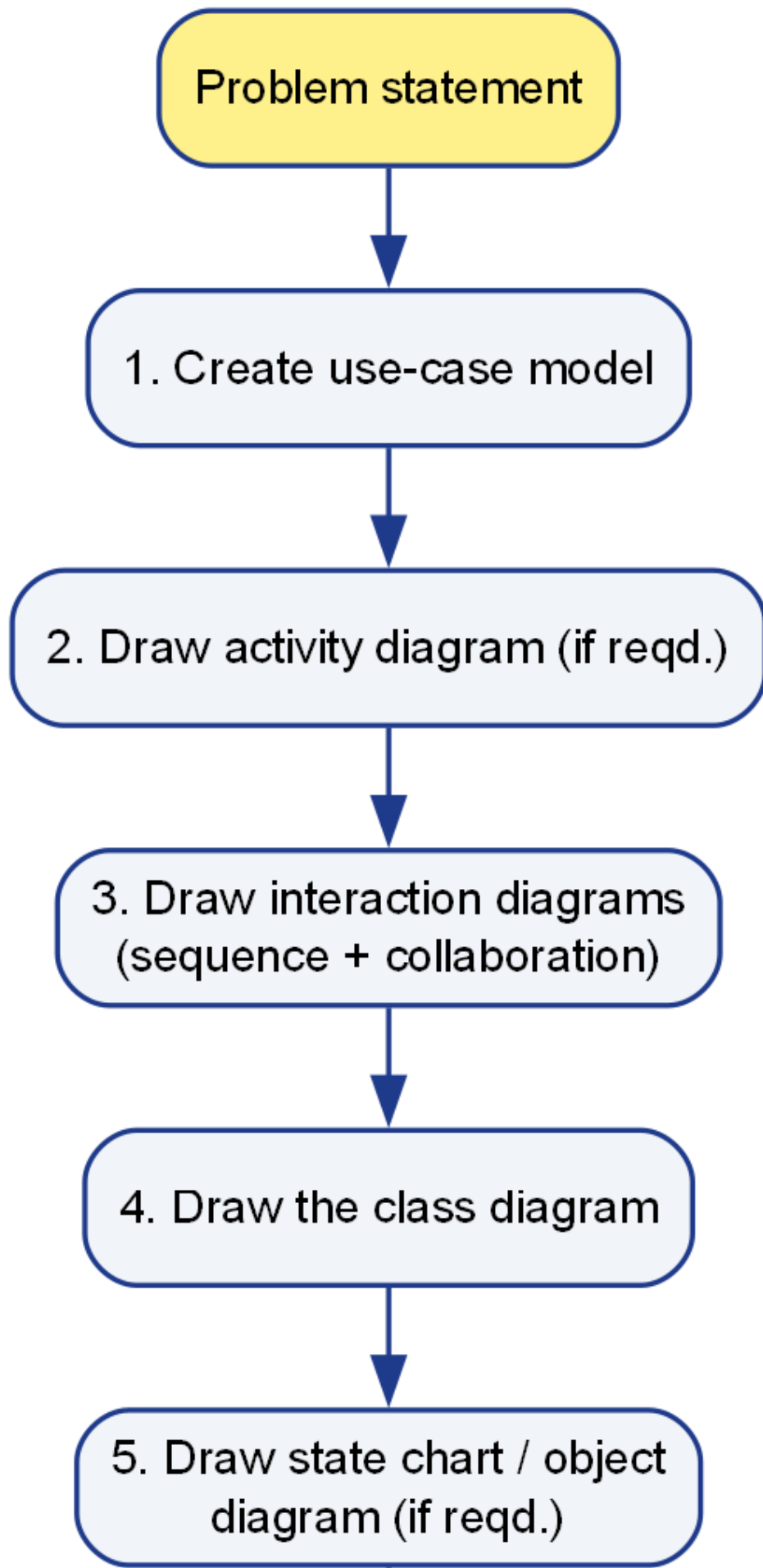
ii. **Draw activity diagram (if required).** An activity diagram illustrates the dynamic nature of a system by modelling the flow of control from activity to activity; an activity represents an operation on some class in the system that results in a change in the state of the system [S:73, Fig.26].

iii. **Draw the interaction diagram.** An interaction diagram shows an interaction consisting of a set of objects and their relationships, including the messages that may be dispatched among them; interaction diagrams address the dynamic view of a system [S:75]. The source's steps are: (a) identify the objects with respect to every use case; (b) draw the sequence diagrams for every use case; (d) draw the collaboration diagrams for every use case [S:75]. The object types used in this analysis model are entity objects, interface objects, and control objects [S:76, Fig.27].

iv. **Draw the class diagram.** The class diagram shows the relationship amongst classes, with four types of relationship [S:77–78]: **association** — a semantic connection between classes, which can be bi-directional or unidirectional [S:77]; **dependencies** — which connect two classes, are always unidirectional, and show that one class depends on the definitions in another [S:78]; **aggregations** — a stronger form of association, a relationship between a whole and its parts [S:78]; and **generalizations** — used to show an inheritance relationship between two classes [S:78].

v. **Design of state-chart diagrams.** A state chart diagram is used to show the state space of a given class, the event that causes a transition from one state to another, and the action that results from a state change; the source gives a state transition diagram for a "book" in the library system [S:79, Fig.28].

vi. **Draw component and deployment diagram.** Component diagrams address the static implementation view of a system and are related to class diagrams in that a component typically maps to one or more classes, interfaces or collaborations; a deployment diagram captures the relationship between physical components and the hardware [S:80].



13. Case Study: University Library Automation [S:81–83]

The source's worked case is **software for automating the manual library of a university**, described as a **stand-alone** system [S:81]. It has three functional areas.

13.1 Issue of books [S:81–82]

A student of any course should be able to get books issued; **books from the General Section are issued to all, but Book Bank books are issued only for their respective courses**; a limitation is imposed on the number of books a student can issue, specifically a **maximum of 4 books from the Book Bank and 3 books from the General section, for 15 days only**; the software **takes the current system date as the date of issue and calculates the date of return** [S:81]. A bar-code detector is used to save the student as well as **book information**, and the due date for return of the book is stamped on the book [S:82].

13.2 Return of books [S:82]

Any person can return the issued books; the student information is displayed using the bar-code detector; the system **displays the student details on whose name the books were issued, as well as the date of issue and return**; the operator **verifies the duration for the issue**; and the information is saved with the **corresponding update in the database** [S:82].

13.3 Query processing and security [S:83]

The system should provide information such as the **availability of a particular book**, the **availability of books by a particular author**, and the **number of copies available of the desired book** [S:83]. It should **generate reports** on the books available at any given time and **printouts for each entry (issue/return)**; security provisions such as **login authenticity** should be provided, with **each user having a user id and a password**; a **record of the users should be kept in a log file**; and **provision should be made for a full backup of the system** [S:83].

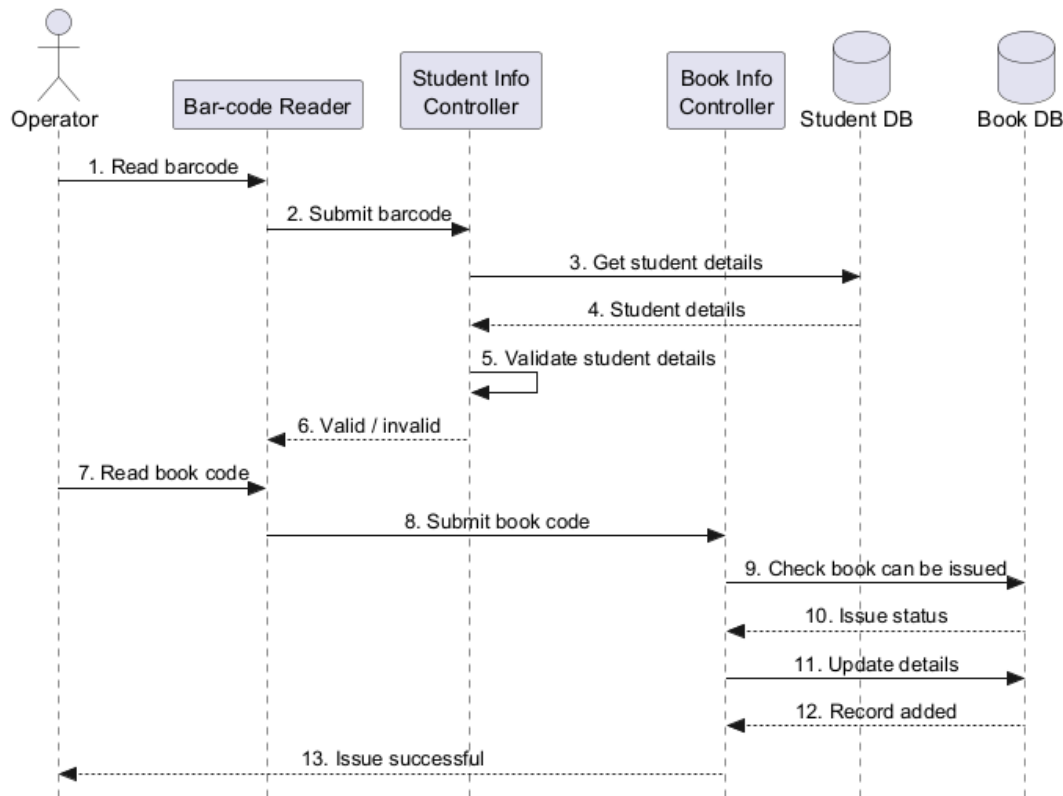


Figure 11. Sequence diagram for the "issue book" scenario of the library case [S:86, modelled from S:81–83].

14. Multiple-Choice Questions and Source Answers [S:94–96]

The source prints twelve multiple-choice questions (5.1–5.12) but does **not** print an answer key [S:94–96]. The answers below are derived **only** from the source's own orderings in Figure 7 (coupling, worst→best) [S:19] and Figure 11 (cohesion, weakest→strongest) [S:27] and the source definitions [S:20–30]; each answer is therefore traceable, but it is an inference from the source ordering, not a printed key.

Q	Question (source wording)	Answer	Source basis
5.1	The most desirable form of coupling is...	(b) Data coupling	best end of Fig.7 [S:19]; definition [S:20]
5.2	The worst type of coupling is...	(a) Content coupling	worst end of Fig.7 [S:19]; definition [S:23]
5.3	The most desirable form of cohesion is...	(c) Functional cohesion	best end of Fig.11 [S:27]; definition [S:28]
5.4	The worst type of cohesion is...	(b) Coincidental cohesion	weakest end of Fig.11 [S:27]; definition [S:30]
5.5	Which one is not a strategy for design?	(c) Embedded design	strategies are bottom-up, top-down, hybrid [S:32–35]
5.6	Temporal cohesion means...	(c) Cohesion with respect to time	definition [S:29]
5.7	Functional cohesion means...	(a) Operations are part of a single functional task and are placed in same procedures	definition [S:28]
5.8	When two modules refer to the same global data area, they are	(d) Common coupled	definition of common coupling [S:21]

	related as...		
5.9	The module in which instructions are related through flow of control is...	(c) Procedural cohesion	definition [S:29]
5.10	The relationship of data elements in a module is called...	(b) Cohesion	definition [S:25]
5.11	A system that does not interact with external environment is called...	(a) Closed system	general systems terminology in source MCQ set [S:96]
5.12	The extent to which different modules are dependent upon each other is called...	(a) Coupling	definition [S:15]

15. Exercises in the Source [S:97–99]

The source's exercises are reproduced here as the authoritative question list for this chapter [S:97–99]:

#	Exercise (source wording)	Covered in section
5.1	What is design? Describe the difference between conceptual design and technical design.	§1, §3
5.2	Discuss the objectives of software design. How do we transform an informal design to a detailed design?	§2, §4.2
5.3	Do we design software when we "write" a program? What makes software design different from coding?	§1
5.4	What is modularity? List the important properties of a modular system.	§5.1–5.2
5.5	Define module coupling and explain different types of coupling.	§6
5.6	Define module cohesion and explain different types of cohesion.	§7
5.7	Discuss the objectives of modular software design. What are the effects of module coupling and cohesion?	§6.6, §8
5.8	If a module has logical cohesion, what kind of coupling is this module likely to have with others?	§7.3, §6
5.9	What problems are likely to arise if two modules have high coupling?	§6.1, §8
5.10	What problems are likely to arise if a module has low cohesion?	§7, §8
5.11	Describe the various strategies of design. Which design strategy is most popular and practical?	§9
5.12	If some existing modules are to be re-used in building a new system, which design strategy is used and why?	§9.2, §9.4
5.13	What is the difference between a flow chart and a structure chart?	§10.4
5.14	Explain why it is important to use different notations to describe software	§10.3

	designs.	
5.15	List a few well-established function-oriented software design techniques.	§10.3
5.16	Define the following terms: Objects, Message, Abstraction, Class, Inheritance and Polymorphism.	§12.2
5.17	What is the relationship between abstract data types and classes?	§12.2
5.18	Can we have inheritance without polymorphism? Explain.	§12.2
5.19	Discuss the reasons for improvement using object-oriented design.	§12.1
5.20	Explain the design guidelines that can be used to produce "good quality" classes or reusable classes.	§12.2
5.21	List the points of a simplified design process.	§12.3
5.22	Discuss the differences between object-oriented and function-oriented design.	§10, §12.1
5.23	What documents should be produced on completion of the design phase?	§11
5.24	Can a system ever be completely "decoupled"?	§6

16. Retrieval Check (answers intentionally withheld)

Answer each from memory, then verify against the anchored section. These are the questions the chapter's own exercises and MCQs make examinable.

1. State the one-word question the source uses to separate design from analysis, and name the design's output document. (§1)
2. List the seven activities of the design framework in order, and name the two feedback steps. (§2)
3. Give any four questions that conceptual design answers, and any four items technical design describes. (§3)
4. State the four required qualities of a design (§4.1) and the four stages from informal outline to finished design (§4.2).
5. List the six properties of a modular system from the source. (§5.2)
6. What does the "modularity and software cost" figure plot, and what region does it mark? (§5.4)
7. Order the six coupling types from worst to best, and define stamp coupling and control coupling in one line each. (§6.4–6.5)
8. In the student-record example, what exactly is passed in the tight design versus the loose design? (§6.3)
9. Order the seven cohesion ladder levels weakest to strongest, and give the source definition of temporal and of procedural cohesion. (§7.2–7.3)
10. What are the source's three reasons that bottom-up design is essential to a top-down approach? (§9.4)
11. List the four function-oriented design notations, and the four functional procedure layers (0–3) with one item each. (§10.3, §10.7)
12. Name the ten design-entity attributes of IEEE 1016, and the four SDD design views with their entity attributes. (§11.2–11.3)
13. Define object, class, encapsulation, and polymorphism using the source's wording. (§12.2)

14. List the six steps for analysing and designing an object-oriented system, and name the four class-diagram relationship types. (§12.3)
15. State the issue limits of the library case (books per section, loan period) and the three query outputs required. (§13)

17. Coverage Map (term → source anchor)

Every major term in this document and the exact slide where the source supports it. This is the claim-to-evidence ledger: any term without a row here does not belong in the notes.

Term	Source anchor
Design as "How", problem-solving, SDD	S:2
Design serves customer + developers	S:4
Design framework activities	S:3, Fig.1
Conceptual vs technical design	S:5, Fig.2; S:6; S:7
Design qualities (correct, understandable, right level, maintainable)	S:8
Informal → detailed design	S:9, Fig.3
Module definitions (range)	S:10
Modular system definition	S:11
Six properties of a modular system	S:12
Modularity = intellectually manageable	S:13
Modularity and software cost	S:14, Fig.4
Coupling definition	S:15
Uncoupled / loosely / highly coupled	S:15–16, Fig.5
Four ways to reduce coupling	S:17
Student-record coupling example	S:18, Fig.6
Coupling types ordering	S:19, Fig.7
Data, stamp coupling	S:20
Control, common coupling	S:21
Common coupling example	S:22, Fig.8
Content coupling	S:23–24, Fig.9
Cohesion definition	S:25, Fig.10
Cohesion types list	S:26
Cohesion ladder	S:27, Fig.11
Functional, sequential cohesion	S:28
Procedural, temporal cohesion	S:29
Logical, coincidental cohesion	S:30
Cohesion/coupling relationship	S:31, Fig.12
Strategy of design rationale	S:32
Bottom-up design	S:33, Fig.13
Top-down design / stepwise refinement	S:34
Hybrid design reasons	S:35
Function-oriented design definition	S:36
Top-down vs reusable structure	S:38–39, Fig.14–15
Design notations	S:40
Structure chart / black box	S:41, Fig.16
Structure-chart notations	S:42, Fig.17
Update-file structure chart	S:43, Fig.18
Transaction-centred structure	S:44–45, Fig.19
Pseudocode	S:46

Functional procedure layers 0–3	S:47–49
IEEE 1016 SDD; scope/references	S:50
Design entity / view / attributes / SDD definitions	S:51
Purpose of SDD; information content	S:52
Ten entity attributes	S:53
SDD organisation + design views	S:54–57
Object-oriented design definition	S:58
Behaviour and state	S:59
Objects	S:60
Messages, abstraction	S:61
Class	S:62, Fig.20–21
Attributes, operations	S:65, Fig.21
Inheritance	S:66–68, Fig.22–23
Polymorphism, encapsulation, hierarchy	S:69, Fig.24
OO steps (use case → deployment)	S:71–80, Fig.25–28
Use case model	S:73
Activity diagram	S:73–74, Fig.26
Interaction diagram; object types	S:75–76, Fig.27
Class diagram; four relationships	S:77–78
State-chart diagram	S:79, Fig.28
Component & deployment diagrams	S:80
Library case: issue / return / query	S:81–83
MCQ 5.1–5.12	S:94–96
Exercises 5.1–5.24	S:97–99

References

16. K. K. Aggarwal and Y. Singh, *Software Engineering*, 3rd ed. New Delhi, India: New Age International Publishers, 2007, ch. 5 (Software Design). — the source of this document (companion slide deck, 99 slides).
17. IEEE, *IEEE Recommended Practice for Software Design Descriptions*, IEEE Std 1016-1998. — cited by the source at [\[S:50–51\]](#).
18. IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998. — referenced by the source at [\[S:50\]](#).
19. IEEE, *IEEE Standard Glossary of Software Engineering Terminology*, IEEE Std 610.12-1990. — referenced by the source at [\[S:50\]](#).